UNIFAAT 2026.2

03 - Terminal e Linguagem de Máquina

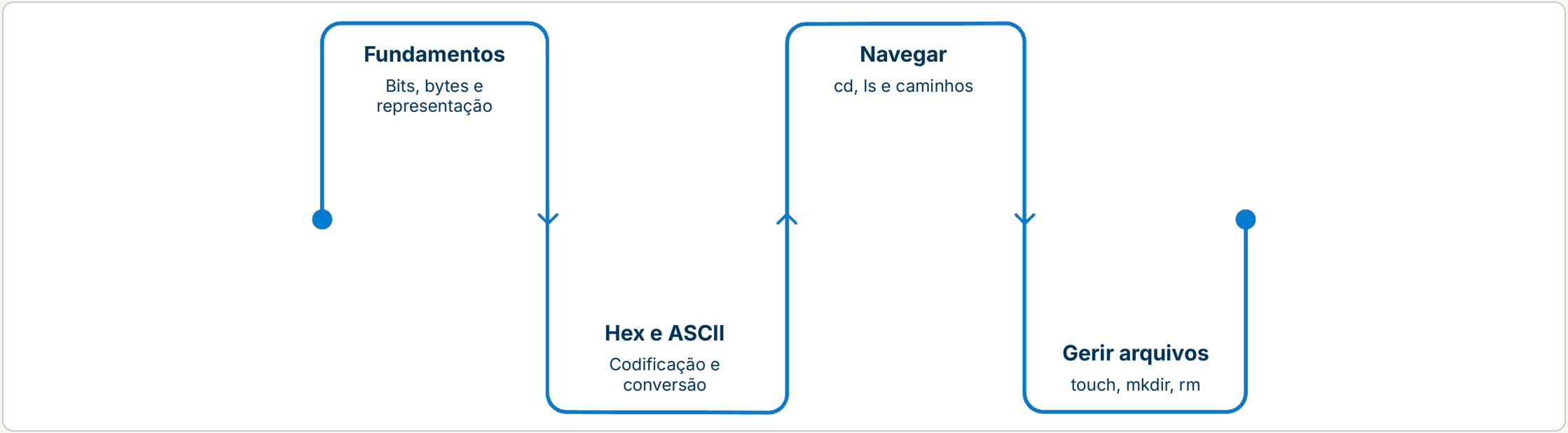
Prof. Luan Lourenço

Cronograma do semestre

AULA	DATA DA AULA	CONTEÚDO
1	19/AGO	Sistema Operacional - GUI e CLI
2	26/AGO	WSL 2 - Virtualização Linux
3	02/SET	Terminal e Linguagem da Máquina
4	09/SET	Versionamento e Git
5	16/SET	Git e GitHub
6	23/SET	Revisão
	30/SET	PROVA 1º BIMESTRE
7	07/OUT	Fundamentos de Python
8	14/OUT	Estrutura de Dados
9	21/OUT	Paradigma Orientado a Objetos
10	28/OUT	OOP - Os 3 Pilares
11	04/NOV	Object Calisthenics
12	11/NOV	Arquitetura de Código
13	18/NOV	Ramificação de Carreira
14	25/NOV	Revisão
	02/DEZ	PROVA 2º BIMESTRE
	09/DEZ	PROVA SUB
	16/DEZ	EXAME

Bits, Bytes e o Terminal Linux

O que o computador realmente enxerga — e como navegar nele pelo terminal.



Por que tudo vira 0 e 1?

```
# Física do processador
Um processador é um circuito elétrico. Só dois estados são possíveis:
Ligado → 1
Desligado → 0
```

```
# Consequência direta
Texto, imagem, som, programa — tudo precisa ser traduzido para sequências de 0s e 1s.
```

Esse sistema se chama **sistema binário**. Cada estado individual é um **bit** — contração de *binary digit*.

Bit, Byte e Octeto

```
# Bit
A menor unidade de informação. Vale 0 ou 1, nunca outra coisa.
```

```
# Byte
Conjunto de 8 bits tratados como uma única unidade. O bloco fundamental do armazenamento.
```

```
# Octeto
Sinônimo formal de byte, muito usado em redes (ex.: cada número de um IPv4 é um octeto).
```

Convertendo Binário ↔ Decimal

Cada posição de um byte representa uma potência de 2. Some apenas as posições onde o bit vale **1**:

tabela de posições

Posição	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
Valor	128	64	32	16	8	4	2	1	—
00001010	**0**	**0**	**0**	**0**	**1**	**0**	**1**	**0**	**10**

8 + 2 = **10 em decimal**. O caminho inverso: encaixe as maiores potências de 2 possíveis até sobrar zero.

Hexadecimal: Atalho para o Binário

```
# Por que base 16?
16 = 2⁴ → cada dígito hex representa exatamente 4 bits (um nibble).
0–9 cobrem as dez primeiras combinações
A–F cobrem as seis restantes (A=10 … F=15)
Um byte inteiro cabe em dois dígitos hex
```

nibbles

Representação	1º Nibble	2º Nibble
Binário	**0100**	**0001**
Hexadecimal	**4**	**1**

Exemplo: `01000001` em binário = `41` em hex = `65` em decimal = letra "A"

Cores em CSS usam hex: `#FF0000` é vermelho puro — dois dígitos por canal R, G, B.

Por que 256 valores em um byte?

Análise Combinatória — quantas sequências diferentes é possível formar com **p** posições, cada uma com **n** opções?

n^p

$$n^p = 2^8 = 256$$

- A contagem começa em **0** (`00000000` já é válido)
- Logo o intervalo vai de **0 a 255**, não de 1 a 256

```
# bits por byte
8
```

```
# combinações possíveis
256
```

```
# intervalo de um byte
0–255
```

A Tabela ASCII

O mesmo byte pode ter significados diferentes — depende de como o programa o **interpreta**.

```
# Como funciona
ASCII define qual caractere corresponde a cada valor de 0 a 255. O byte 01000001 pode ser:
O número 65 (decimal)
A letra A (caractere)
```

ascii

Caractere	Decimal	Binário
A	65	01000001
a	97	01100001
0	48	00110000
espaço	32	00100000

i A diferença entre "A" (65) e "a" (97) é exatamente **32** — padrão da tabela ASCII, explorável em código.

De Byte a Gigabyte

Os múltiplos seguem potências de 2 — por isso um "GB" computacional não é exatamente 1 bilhão de bytes.

unidades

Unidade	Equivalência	Em bytes
1 Byte	8 bits	1
1 Kilobyte	1.024 bytes	2^{10}
1 Megabyte	1.024 KB	2^{20} = 1.048.576
1 Gigabyte	1.024 MB	2^{30} = 1.073.741.824

! Pen drive "de 16 GB" mostra menos no sistema: fabricante usa base 10; o SO usa base 2.

Do Binário ao Terminal

Diretório vs. Pasta
Diretório é o nome técnico de "pasta" no mundo Linux. Mesma coisa — só vocabulário diferente.

Cada comando é um binário
Ao digitar `ls` ou `mkdir`, o terminal localiza e executa um **arquivo binário compilado** com aquele nome.

Recursividade
Uma operação **recursiva** se aplica a algo e se repete em cada parte menor, descendo nível por nível — até não sobrar nada.

i `cd` é exceção: é um comando *built-in* do shell, pois precisa alterar o estado do próprio terminal.

Navegando pelo Terminal

~

```
# pwd: mostra o caminho completo do diretório atual. Primeiro comando para "se situar".
$ pwd
# -l mostra detalhes; -a inclui arquivos ocultos (começam com .).
$ ls
$ ls -la
# entra no diretório
$ cd nome
# sobe um nível
$ cd ..
# sozinho, vai para o home
$ cd
```

Criando e Lendo Arquivos

~

```
# cria um diretório
$ mkdir nome
# com -p, cria toda a estrutura aninhada de uma vez
$ mkdir -p projeto/src/utils
# cria um arquivo vazio; se já existir, apenas atualiza a data de modificação
$ touch arquivo.txt
# exibe o conteúdo de um arquivo diretamente no terminal — ideal para leitura rápida
$ cat arquivo.txt
```

Removendo com Cuidado: `rm`

> ! **Sem lixeira.** O terminal **não pergunta "tem certeza?"** e não usa lixeira — o arquivo some definitivamente. `rm -r` pode apagar centenas de arquivos em silêncio. Use com atenção redobrada.

~

```
$ rm arquivo.txt
$ rm diretório
$ rm -r diretório
```

Isso é consequência direta de o terminal falar direto com o sistema, sem a camada extra da GUI.

Outros Comandos Essenciais

~

```
# copia um arquivo ou diretório, mantendo o original intacto
$ cp origem destino
# move ou renomeia um arquivo — renomear é tecnicamente mover para o mesmo lugar com outro nome
$ mv antigo novo
# abre o manual oficial de qualquer comando diretamente no terminal. A melhor documentação disponível
$ man comando
```

tarefa

TF 03

Entrega pelo formulário: `docs.google.com/forms/d/e/1FAIpQLSd8cZIMKwLPuqLB0EddWjUKr6epZg-_vFB6D3TdShuhFSMp5Q/viewform`